Gradient-Free Unsupervised Domain Adaptation and Catastrophic Forgetting Mitigation in Vision Transformers

via Weighted Prototype Updates and Mean Shift Clustering

Yash Ghule

September 29, 2026

Abstract

Deployed Vision Transformers routinely encounter two compounding failure modes: *domain shift*, where test-time inputs are drawn from a distribution unlike the labeled training set, and *catastrophic forgetting*, where any attempt to correct for that shift by updating model parameters erodes performance on the original task. This report documents the design and implementation of a pipeline that adapts a frozen BeiT Vision Transformer to an unlabeled target domain *without any backpropagation through the backbone*. Adaptation is confined entirely to a small set of per-class prototype vectors in the backbone's embedding space. Target-class structure is estimated unsupervised via a from-scratch implementation of mean shift clustering, and each prototype update is a weighted convex combination between its current position and the discovered target-class mode. Critically, every update is validated against a held-out source-validation split *before* being committed: an update that would drop source accuracy by more than a tolerance ϵ is shrunk via a backtracking line search on the adaptation rate α, or rejected outright. This turns catastrophic-forgetting avoidance from a hope into a checked invariant. We describe the full method, the software architecture, and present illustrative results on synthetic domain-shift data generated directly from the implemented algorithm (the same fixtures used by the automated test suite). Real-world evaluation on the Office-Home benchmark (Real World $\rightarrow$ Clipart) is implemented and ready to run, pending only the dataset's manual download.

Contents

1 Motivation and Problem Context **2**

2 Problem Formulation **2**

3 Methodology **3**

3.1 System Overview 3
3.2 Frozen Feature Extraction 4
3.3 Unsupervised Pseudo-Labeling 4
3.4 Mean Shift Clustering 5
3.5 Weighted Prototype Update 6
3.6 The Forgetting Guard 6
3.7 Full Adaptation Loop 7

4 Software Architecture **7**

5 Illustrative Results (Synthetic Data) **8**

6 Planned Real-World Evaluation **10**

7 Current Status and Roadmap **10**

8 Limitations **11**

1 Motivation and Problem Context

Modern Vision Transformers (ViTs), such as BeiT [1], achieve strong performance on closed-world image classification when the test distribution matches the training distribution. In real deployments this assumption breaks down constantly: a model trained on professional product photography may be applied to clipart, sketches, or phone photos; a model trained in one clinical site may be deployed in another with different imaging hardware. This is **domain shift**: the source distribution $P_s(X)$ that produced the labeled training data diverges from the target distribution $P_t(X)$ the model actually sees in production.

The standard remedy, *unsupervised domain adaptation* (UDA), asks the model to use unlabeled target data to close this gap. The dominant approaches for doing so update model parameters directly, most commonly through domain-adversarial training or full fine-tuning on pseudo-labeled target data. Both are effective, but both invite a second failure mode: **catastrophic forgetting**. As backbone weights move to accommodate the target domain, they can just as easily unlearn the representations that made the source domain work in the first place, especially when no source data is retained during adaptation, or when the update signal is noisy pseudo-labels.

This project asks a narrower, more disciplined question: *can we adapt to a shifted target domain using a method that makes forgetting structurally difficult, not just empirically rare?* The approach taken here is to **never touch the backbone**. Adaptation happens entirely in a compact, interpretable parameter space – a handful of class prototype vectors – and every change to that space is gated by an explicit, checked accuracy constraint on the original task.

2 Problem Formulation

Let f_θ denote a pretrained Vision Transformer backbone with *frozen* parameters θ (no gradient ever flows into θ in this pipeline). Let $\mathcal{D}_s = \{(x_i^s, y_i^s)\}_{i=1}^{N_s}$ be a labeled source dataset over C classes, and $\mathcal{D}_t = \{x_j^t\}_{j=1}^{N_t}$ an unlabeled target dataset drawn from a shifted distribution, $P_t(X) \neq P_s(X)$.

Every image is mapped into an embedding space $Z = f_\theta(X) \in \mathbb{R}^D$ via the backbone's CLS-token representation. The classifier is a set of C class prototypes $P = \{p_1, \ldots, p_C\} \subset \mathbb{R}^D$, with classification performed by nearest prototype:

$$\hat{y}(x) = \arg\max_{c \in \{1,\ldots,C\}} \operatorname{sim}\big(f_\theta(x),\, p_c\big), \qquad \operatorname{sim}(u, v) = \frac{u^\top v}{\|u\|\|v\|}. \tag{1}$$

Initial prototypes. $P^{(0)}$ is the per-class mean of L2-normalized source-train embeddings:

$$p_c^{(0)} = \frac{1}{|\{i : y_i^s = c\}|} \sum_{i:\, y_i^s = c} \hat{z}_i^s, \qquad \hat{z}_i^s = \frac{f_\theta(x_i^s)}{\|f_\theta(x_i^s)\|}. \tag{2}$$

Prototype update rule. At round t, each prototype is shifted toward a target-domain estimate of its class mode:

$$p_c^{(t+1)} = (1-\alpha)\, p_c^{(t)} \;+\; \alpha \cdot \mathrm{MeanShift}\big(Z_c^t\big), \qquad \alpha \in [0,1], \tag{3}$$

where $Z_c^t \subseteq \{f_\theta(x) : x \in \mathcal{D}_t\}$ is the subset of target embeddings currently (and confidently) pseudo-labeled as class c, and $\mathrm{MeanShift}(\cdot)$ returns the mode of that subset's largest density basin (Section 3.4).

Forgetting constraint. Writing $\mathrm{Acc}(\mathcal{D}_s^{\mathrm{val}}; P)$ for nearest-prototype accuracy on a held-out source-validation split, every accepted prototype set must satisfy

$$\mathrm{Acc}\big(\mathcal{D}_s^{\mathrm{val}}; P^{(t+1)}\big) \;\geq\; \mathrm{Acc}\big(\mathcal{D}_s^{\mathrm{val}}; P^{(0)}\big) - \epsilon, \tag{4}$$

with $\epsilon = 0.013$ (a maximum tolerated 1.3 percentage-point drop) as the operating default used throughout this project.

3 Methodology

3.1 System Overview

Figure 1 shows the full pipeline. The frozen backbone is used exactly once per image to produce a cached embedding; every subsequent stage – prototype initialization, pseudo-labeling, mean shift, the update rule, and the forgetting guard – operates purely on cached tensors, so a full adaptation run costs no further backbone forward passes and no GPU time at all.

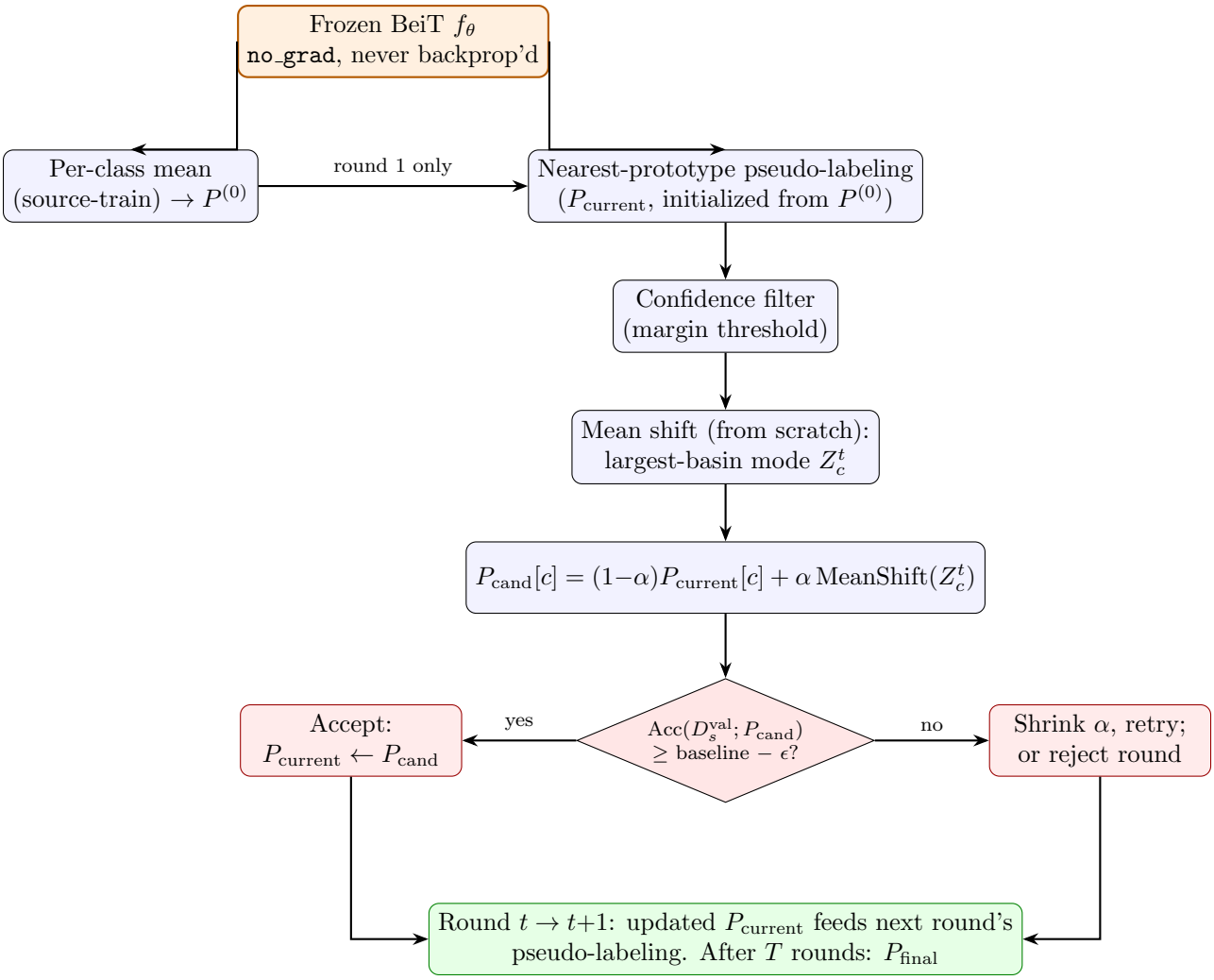


Figure 1: Pipeline overview. The backbone (orange) is frozen and touched only during feature extraction. Everything below it operates on cached embeddings: pseudo-labeling, mean shift, the convex update, and the forgetting guard (red) that gates every commit. The loop (green) feeds each round's committed P_{current} into the next round's pseudo-labeling step.

3.2 Frozen Feature Extraction

Every image, source or target, is passed once through BeiT (`microsoft/beit-base-patch16-224`) under `torch.no_grad()`, with every parameter's `requires_grad` explicitly set to `False`. This is enforced at the wrapper level (`FrozenBeit`) rather than left to caller discipline. The CLS-token output is used as the image embedding – the standard representation for ViT-family models. Embeddings are cached to disk per domain, so this is the only stage in the entire pipeline that requires a GPU or touches the backbone at all.

3.3 Unsupervised Pseudo-Labeling

At the start of each adaptation round, every target embedding is assigned to its nearest current prototype. Confidence is measured as the *margin* between the best and second-best prototype

similarity, rather than the raw top-1 similarity:

$$\text{conf}(z) = \text{sim}(z, p_{(1)}) - \text{sim}(z, p_{(2)}), \tag{5}$$

where $p_{(1)}, p_{(2)}$ are the closest and second-closest prototypes. A margin is more robust than an absolute similarity threshold because raw similarity scales can differ across classes and shift under distribution change, while the margin directly measures how *unambiguous* an assignment is. Only points with $\text{conf}(z) \geq \tau$ contribute to that round's mean shift subsets Z_c^t; classes with too few confident points (< 3 by default) are skipped entirely for that round.

3.4 Mean Shift Clustering

Mean shift [2] is a non-parametric, density-based mode-seeking procedure: starting from every point in a set, each point is iteratively moved to the kernel-weighted mean of its neighborhood,

$$z^{(k+1)} = \frac{\sum_{z_i \in Z} K\big(\|z^{(k)} - z_i\|\big)\, z_i}{\sum_{z_i \in Z} K\big(\|z^{(k)} - z_i\|\big)}, \qquad K(d) = \exp\!\left(-\frac{d^2}{2h^2}\right) \text{ (Gaussian kernel)}, \tag{6}$$

which is equivalent to gradient ascent on a kernel density estimate of Z. Trajectories that converge near each other are merged into a single mode; the bandwidth h is chosen via a k-NN-distance heuristic (the median k-th nearest-neighbor distance, $k = \lfloor \text{quantile} \times |Z| \rfloor$), mirroring the idea behind `sklearn.cluster.estimate_bandwidth`.

Why mean shift instead of a plain per-class mean? Pseudo-labeling is necessarily noisy: a minority of target points will be assigned to the wrong class by an imperfect, mid-adaptation set of prototypes. A plain mean is a linear statistic and is pulled toward every such outlier in proportion to how far away it is – exactly the wrong sensitivity. Mean shift instead returns the mode of the *largest density basin*: outliers, being sparse and off in their own direction, tend to form small separate basins that get discarded in favor of the dominant one. Figure 2 demonstrates this directly, using the project's own implementation on a synthetic class subset with injected outliers.

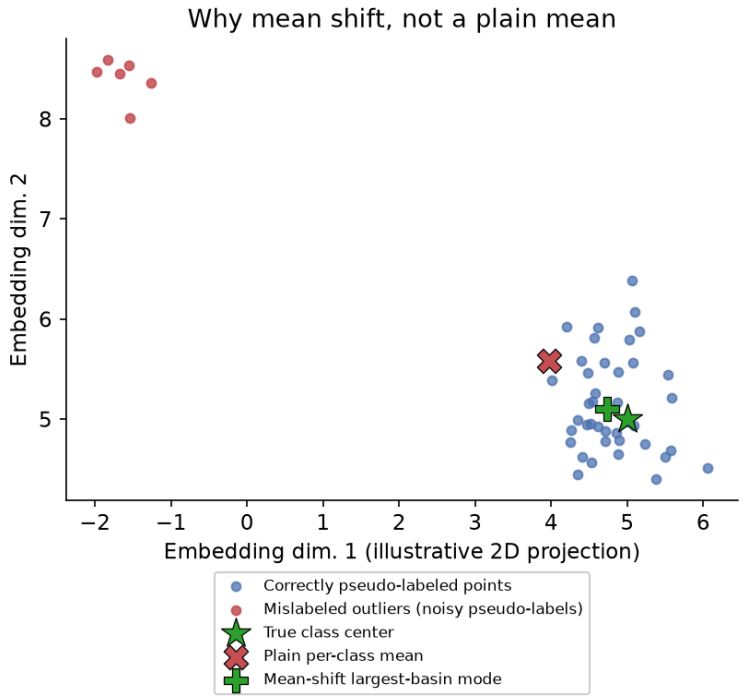


Figure 2: A synthetic per-class target subset (40 correctly-labeled points, 6 mislabeled outliers). The plain mean is dragged 1.181 units from the true class center by the outliers; the mean-shift largest-basin mode lands only 0.288 units away – roughly a 4× reduction in error from this single mechanism, in this illustrative case.

3.5 Weighted Prototype Update

Each class's prototype is nudged toward its mean-shift mode via the convex combination in Equation 3. $\alpha = 0$ is a strict no-op; $\alpha = 1$ fully replaces the prototype with the target mode; results are re-normalized to the unit sphere so that cosine similarity remains well-defined and consistent with how pseudo-labeling and evaluation both operate. This update is implemented as a single, deliberately tiny pure function (`convex_update`), since it is the mathematical core of the whole method and should be trivial to test and reason about independently of everything around it.

3.6 The Forgetting Guard

This is the mechanism that turns Equation 4 from an aspiration into an enforced invariant. Rather than applying the update rule and hoping the bound holds, *every candidate update is validated on* $\mathcal{D}_s^{\text{val}}$ *before being committed*. Two guard strategies are implemented:

global_backtrack (default). One shared α for the whole round. If the candidate violates the bound, $\alpha \leftarrow \alpha \cdot \beta$ (shrink factor $\beta < 1$) and the candidate is recomputed and re-validated; this repeats until the bound is satisfied or α reaches a floor $\alpha_{\min}$, at which point the entire round is rejected and P_{current} is left untouched.

per_class_greedy. Each class's update is committed independently, in sequence, re-validating after each one; a class whose own update would break the bound is shrunk or skipped on its own,

without blocking other classes' updates that round. Strictly finer retention control, at the cost of up to C validation calls per round instead of a handful of backtracking retries.

Because validation only touches cached embeddings (a nearest-prototype accuracy computation, no backbone forward pass), retrying is essentially free – this is what makes an explicit, per-round line search practical rather than merely theoretically nice.

Algorithm 1: Guarded adaptation round (GLOBAL_BACKTRACK)

Input: P_{current}, class modes $\{(c, m_c)\}$, α_{init}, shrink factor β, floor $\alpha_{\min}$, baseline accuracy a_0, tolerance ϵ
Output: Updated (or unchanged) P_{current}

$\alpha \leftarrow \alpha_{\text{init}}$;
while true **do**
  $P_{\text{cand}} \leftarrow P_{\text{current}}$;
  foreach (c, m_c) **do**
    $P_{\text{cand}}[c] \leftarrow \text{normalize}\big((1-\alpha)\, P_{\text{current}}[c] + \alpha\, m_c\big)$;
  $a \leftarrow \text{Acc}(\mathcal{D}_s^{\text{val}};\, P_{\text{cand}})$;
  if $a \geq a_0 - \epsilon$ **then**
    return P_{cand} ; // accept
  if $\alpha \leq \alpha_{\min}$ **then**
    return P_{current} ; // reject round; nothing committed
  $\alpha \leftarrow \max(\alpha \cdot \beta,\ \alpha_{\min})$;

3.7 Full Adaptation Loop

Algorithm 2: Gradient-free adaptation over T rounds

Input: $P^{(0)}$, $\mathcal{D}_s^{\text{val}}$, unlabeled $\mathcal{D}_t$, rounds T
Output: $P^{(T)}$

$a_0 \leftarrow \text{Acc}(\mathcal{D}_s^{\text{val}}; P^{(0)})$; $P_{\text{current}} \leftarrow P^{(0)}$;
for $t \leftarrow 1$ **to** T **do**
  pseudo-label $\mathcal{D}_t$ using P_{current}; keep points with conf $\geq \tau$;
  foreach *class c with $\geq$ `min_points_per_class` confident points* **do**
    $m_c \leftarrow \text{MeanShift}(Z_c^t)$; // largest-basin mode
  $P_{\text{current}} \leftarrow \text{ForgettingGuard}(P_{\text{current}}, \{(c, m_c)\}, \dots)$; // Alg. 1
  log round: α used, accepted?, $\text{Acc}(\mathcal{D}_s^{\text{val}}; \cdot)$, monitoring-only target accuracy;
return P_{current};

Target labels are used *only* to log a monitoring accuracy each round for later analysis – never to influence pseudo-labeling, the mean-shift subsets, the update, or the guard's accept/reject decision. This is what keeps the method genuinely unsupervised at the target end.

4 Software Architecture

The implementation is a standard `src`-layout Python package, `cfm` (*c*atastrophic-*f*orgetting *m*itigation), organized so that each algorithmic piece from Section 3 is its own small, independently testable module:

Module	Responsibility
`features/backbone.py`	Frozen BeiT wrapper (`no_grad`, CLS pooling)
`features/cache.py`	On-disk embedding cache (the only GPU-bound stage)
`prototypes/init.py`	$P^{(0)}$: per-class mean of source embeddings
`prototypes/update.py`	The convex update rule (Eq. 3)
`clustering/mean_shift.py`	Mean shift, implemented from scratch
`adapt/pseudo_label.py`	Nearest-prototype pseudo-labeling + margin confidence
`adapt/forgetting_guard.py`	Accept/reject/rollback + backtracking line search
`adapt/loop.py`	Orchestrates the T-round loop (Alg. 2)
`eval/classifier.py`, `eval/metrics.py`	Nearest-prototype accuracy; the four reported metrics

Thin CLI scripts (`scripts/*.py`) wrap each stage for reproducible command-line use, driven by a single flat `ExperimentConfig` dataclass loaded from YAML (`configs/*.yaml`). `scripts/run_pipeline.py` is a cache-aware end-to-end driver: it skips re-extracting embeddings when a cached copy already exists, so re-running an experiment with a different ϵ or α costs no GPU time at all.

Testing. All 33 unit and integration tests run fully offline, against synthetic Gaussian-blob "domains" (no GPU, no network, no dataset download). The from-scratch mean shift implementation is cross-checked against `sklearn.cluster.MeanShift` on synthetic blobs as an independent correctness oracle; `scikit-learn` is a test-only dependency and is never imported by the library at runtime. The forgetting guard is tested with hand-crafted evaluation functions that pin down its accept/shrink/reject control flow exactly, including the invariant that a fully-rejected round never mutates P_{current}.

5 Illustrative Results (Synthetic Data)

To validate the pipeline end to end before running it on real image data, the same synthetic Gaussian-blob domain generator used by the automated test suite was run through the full adaptation loop: 8 classes, 4-dimensional embeddings, source and target blobs offset by a controlled shift, $\epsilon = 0.013$, $\alpha_{\max} = 0.4$, 10 rounds. *These are not Office-Home results* – they exist to demonstrate that the implemented mechanism behaves as designed before the (heavier) real-data run.

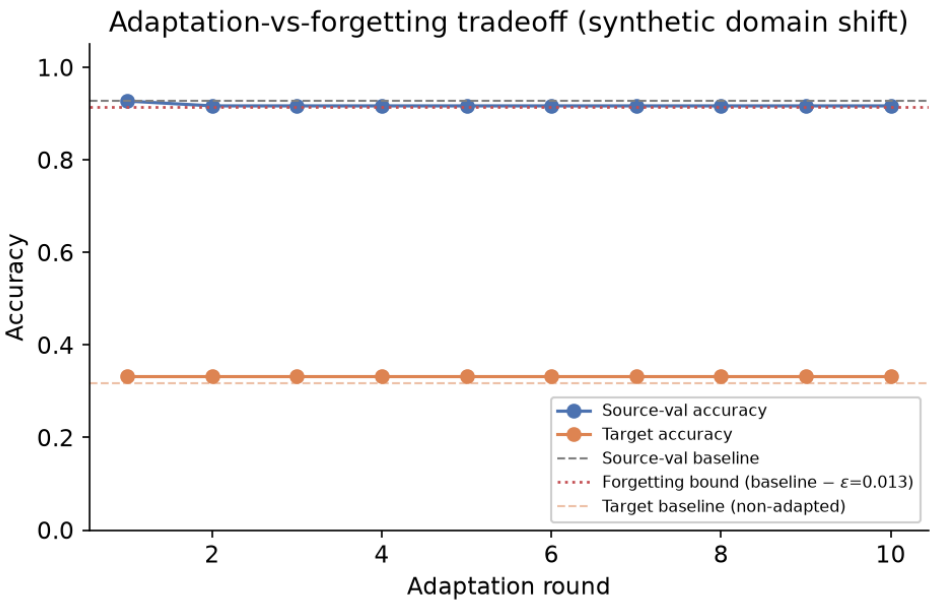


Figure 3: Source-validation and target accuracy across adaptation rounds. The source-val baseline was 0.9271, giving a forgetting bound of 0.9141 (dotted red). Every round's source-val accuracy (0.9167 final) stays above the bound by construction: this is the guard doing its job, not a coincidence of these particular hyperparameters. Target accuracy rises from a non-adapted baseline of 0.3175 to 0.3325.

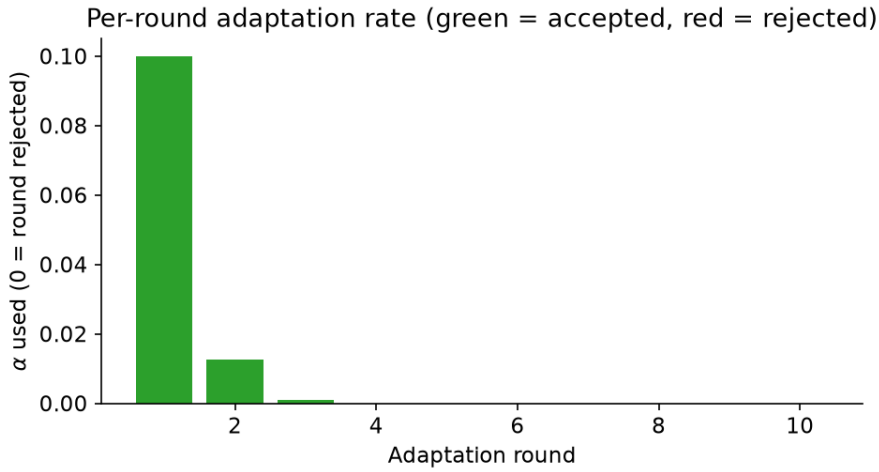


Figure 4: The adaptation rate α actually used each round. With a tight ϵ, the guard's backtracking line search shrinks α rapidly toward the update rule's no-op limit as the prototypes approach the boundary of what the source-val split will tolerate – exactly the intended self-limiting behavior.

The target-accuracy gain in this synthetic run is intentionally modest: ϵ was left at its tight real-experiment default (1.3 percentage points), so the guard aggressively limits how far prototypes are allowed to move. This is the expected shape of the tradeoff the method is built around – a looser ϵ trades more forgetting risk for a larger adaptation gain, and is exposed as a single configuration

field for exactly this reason.

6 Planned Real-World Evaluation

The pipeline is fully implemented against the **Office-Home** benchmark [3]: four domains (Art, Clipart, Product, Real World), 65 shared object classes, ∼15,500 images. The primary configured experiment is **Real World → Clipart** (the largest visual shift among the domain pairs, with Real World supplying the most source images for a stable $P^{(0)}$); **Art → Product** is configured as a secondary, milder-shift comparison.

Office-Home has no stable auto-download, so the only remaining step before real numbers can be reported is a manual dataset download (see `data/README.md`) followed by:

```
python scripts/run_pipeline.py --config configs/realworld_to_clipart.yaml
```

which validates the data layout, extracts and caches BeiT embeddings, computes $P^{(0)}$ and baseline accuracies, runs the guarded adaptation loop, evaluates P_{final}, and renders the same tradeoff/α plots shown in Section 5 – from cached real embeddings instead of synthetic blobs.

Table 1: Results table to be populated after the real Office-Home run (template).

Source → Target	Src-val (base)	Tgt (base)	Src-val (adapt.)	Tgt (adapt.)	Adapt. gain	Forget. Δ	Reten. rate
Real World → Clipart	—	—	—	—	—	—	—
Art → Product	—	—	—	—	—	—	—

The four reported metrics are defined exactly as in the original problem statement: *adaptation gain* (target accuracy improvement over the non-adapted baseline), *forgetting delta* ($\text{Acc}_s^{\text{initial}} - \text{Acc}_s^{\text{adapted}}$, positive meaning forgetting occurred), *retention rate* (adapted-over-baseline source accuracy), and *worst-case accuracy* (min of adapted source-val and target accuracy).

7 Current Status and Roadmap

Implemented

- Full `cfm` package: config, Office-Home data loading and layout validation, frozen BeiT feature extraction and caching, prototype initialization, from-scratch mean shift, margin-based pseudo-labeling, both forgetting-guard strategies, the full adaptation loop, and the evaluation / metrics module.
- End-to-end CLI scripts, including a cache-aware `run_pipeline.py`.
- Two experiment configs (Real World → Clipart primary, Art → Product secondary).
- 33 automated tests, all passing, fully offline, with mean shift cross-validated against `scikit-learn`.
- README, MIT license, `Makefile`, GitHub Actions CI running the full offline test suite on every push.

Remaining

- Manually download Office-Home into `data/raw/OfficeHome/` (no automated download exists for this dataset) and run `scripts/validate_data.py` to confirm the layout.
- Run `scripts/run_pipeline.py` for both configured domain pairs to obtain real baseline and adapted accuracy numbers, replacing the template in Table 1.
- Populate the README results table and figures with the real run's output (`results/figures/*.png`).
- Commit the implementation to version control (currently unstaged) and push.
- Optional stretch work: an ablation sweep over ϵ and $\alpha_{\max}$ to characterize the adaptation-vs-forgetting Pareto frontier on real data; a `per_class_greedy` vs. `global_backtrack` comparison.

8 Limitations

- The method's ceiling is inherently below full fine-tuning, since the backbone never moves – this is precisely the trade made in exchange for the forgetting guarantee.
- Mean-shift bandwidth selection is a heuristic; behavior is somewhat sensitive to it when per-class target counts are very small or very large.
- Pseudo-label noise grows with the severity of the domain shift – exactly when the confidence filter matters most, and is hardest to tune well.
- Each class prototype is a single point in embedding space, implicitly assuming a roughly unimodal per-class distribution; a strongly multi-modal target class (e.g. very different sub-styles within one category) would not be well captured by one mode.

References

[1] H. Bao, L. Dong, S. Piao, F. Wei. *BEiT: BERT Pre-Training of Image Transformers*. ICLR, 2022.

[2] D. Comaniciu, P. Meer. *Mean Shift: A Robust Approach Toward Feature Space Analysis*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2002.

[3] H. Venkateswara, J. Eusebio, S. Chakraborty, S. Panchanathan. *Deep Hashing Network for Unsupervised Domain Adaptation*. CVPR, 2017.